



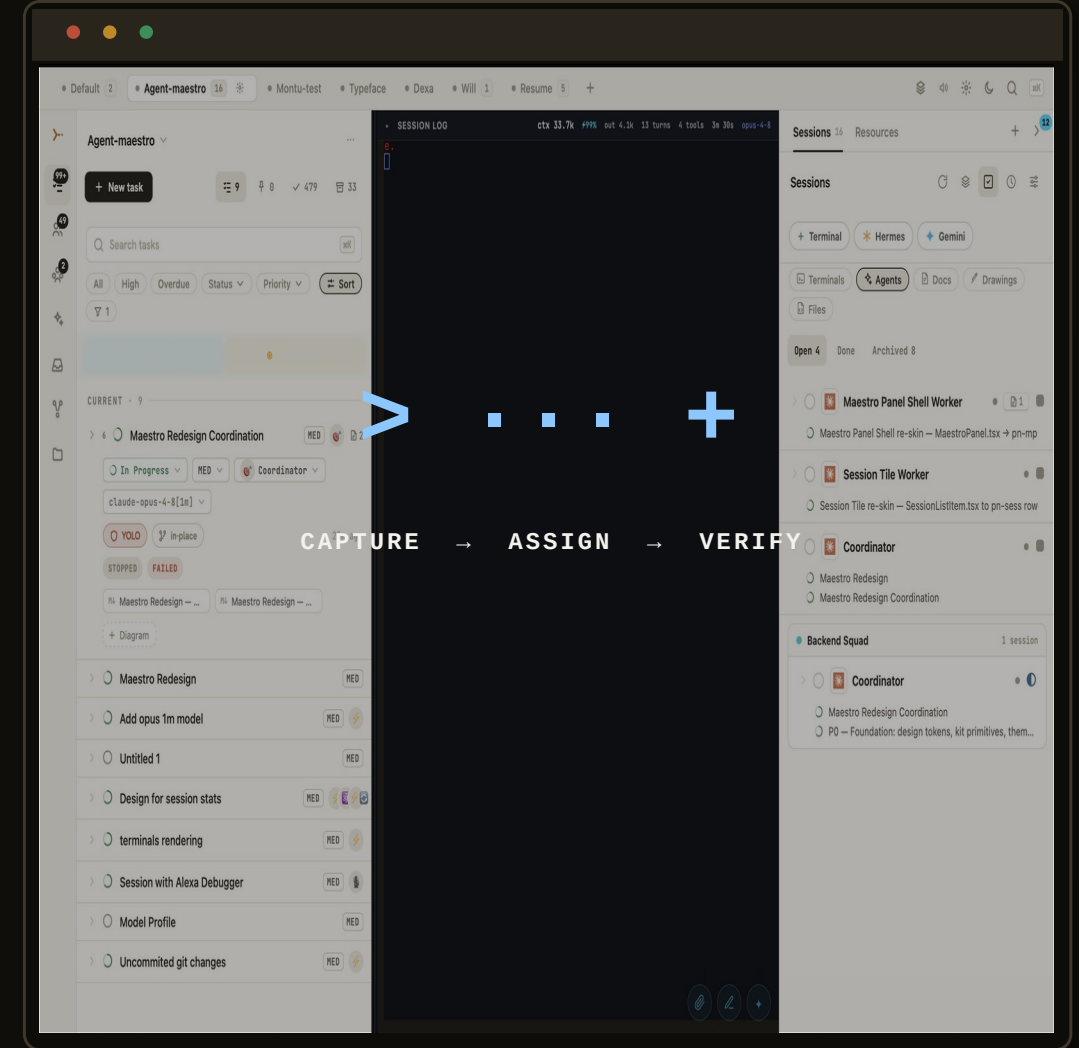
MAESTRO

STRATEGY • JULY 2026

The open control plane for agentic product work

Turn ideas into owned, observable, verified outcomes
across Claude, Codex, Hermes—and whatever comes
next.

SYSTEMS TO BUILD SYSTEMS



The models got agents. Work did not get a system.

The strongest wedge is lived pain—not an invented market problem.

01 You felt the pain

Multiple agents improved output—but multiplied context switching, duplicated work, and review debt.

02 You built a working answer

Maestro changed how ideas become tasks, how agents are assigned, and how progress stays visible.

03 The product is the proof

Maestro is being built through Maestro: task trees, specialist roles, artifacts, peer review, and iteration.

04 The story is authentic

“My process stopped scaling. I built an operating system around it. Now I am opening it to other power users.”

FOUNDER TRANSFORMATION

Agent chaos was costing me time, attention, and trust.

I built a shared workflow for defining, assigning, observing, and verifying work—and now I can move multiple products forward without losing ownership or context.

STRATEGIC THESIS

Do not sell “more agents.” Sell control, continuity, and verified progress.

The visible problem is session chaos. The valuable problem is deeper.

Sell the surface pain. Solve the operating-system gap underneath it.

SURFACE PROBLEM



ROOT PROBLEM

No reliable system turns intent into owned, verifiable, reusable work.

- Ownership and dependencies stay implicit.
- "Agent finished" is confused with "work is correct."
- Good prompts, feedback, and decisions disappear with the session.

TIME

Less context switching and rework

MONEY

Fewer duplicated runs and wasted tokens

CONTROL

Clear ownership, evidence, and continuity

Start with the people who already feel the coordination pain.

The first 100–1,000 users are not “everyone who uses AI.”

PRIMARY BEACHHEAD

Frontier-tool power users

Daily users of Claude Code, Codex, and Hermes who already run several sessions, projects, and worktrees.

- Terminal + Git native
- Model-switching by instinct
- Active on X, GitHub, Discord, Reddit
- Pain is immediate—not educational

SECOND

Solo product builders

Indie hackers and tiny AI-native teams seeking leverage without coordination overhead.

- Many hats
- Outcome driven
- Need repeatability

EXPAND

Free/local entrants

Creators entering through Ollama and local models via mobile and short-form content.

- Cost sensitive
- Needs guided setup
- Broader audience

NOT NOW

Casual chatbot users · enterprise procurement-led buyers · teams seeking only a Jira replacement

Own the workflow layer—not the model layer.

Freedom of tools. Discipline of process.

CATEGORY

Agentic Product Management

The system that coordinates intent, humans, agents, tools, code, evidence, and organizational memory in one delivery loop.



Claude



Codex



Hermes

ANY NEXT RUNTIME

ONE-LINE POSITION

Maestro is the open control plane for agentic product work.

NOT

- another model
- a multi-terminal wrapper
- a generic autonomy framework
- a replacement for GitHub review

The product is the closed loop—not the number of agents.

Software is built progressively. Maestro makes the progression explicit.



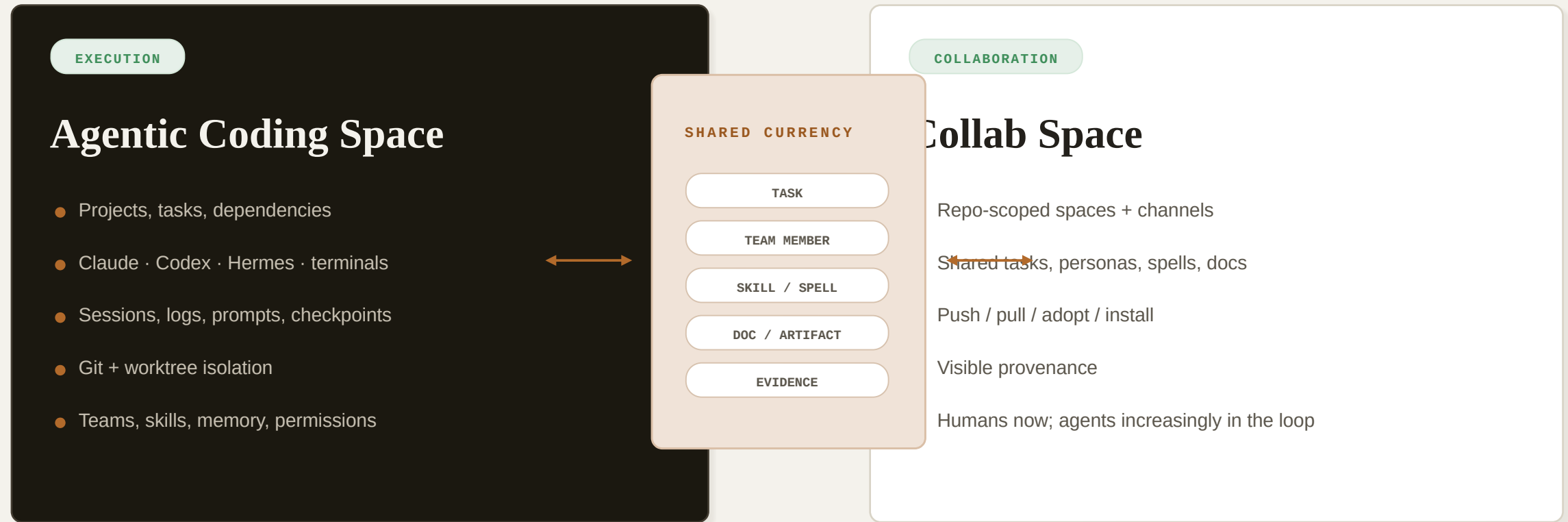
THE REAL AHA

*“I left, came back,
understood what
happened—and
trusted the
result.”*

Spawning agents is an action.
Closing the loop is the product.

Two spaces. One shared work graph.

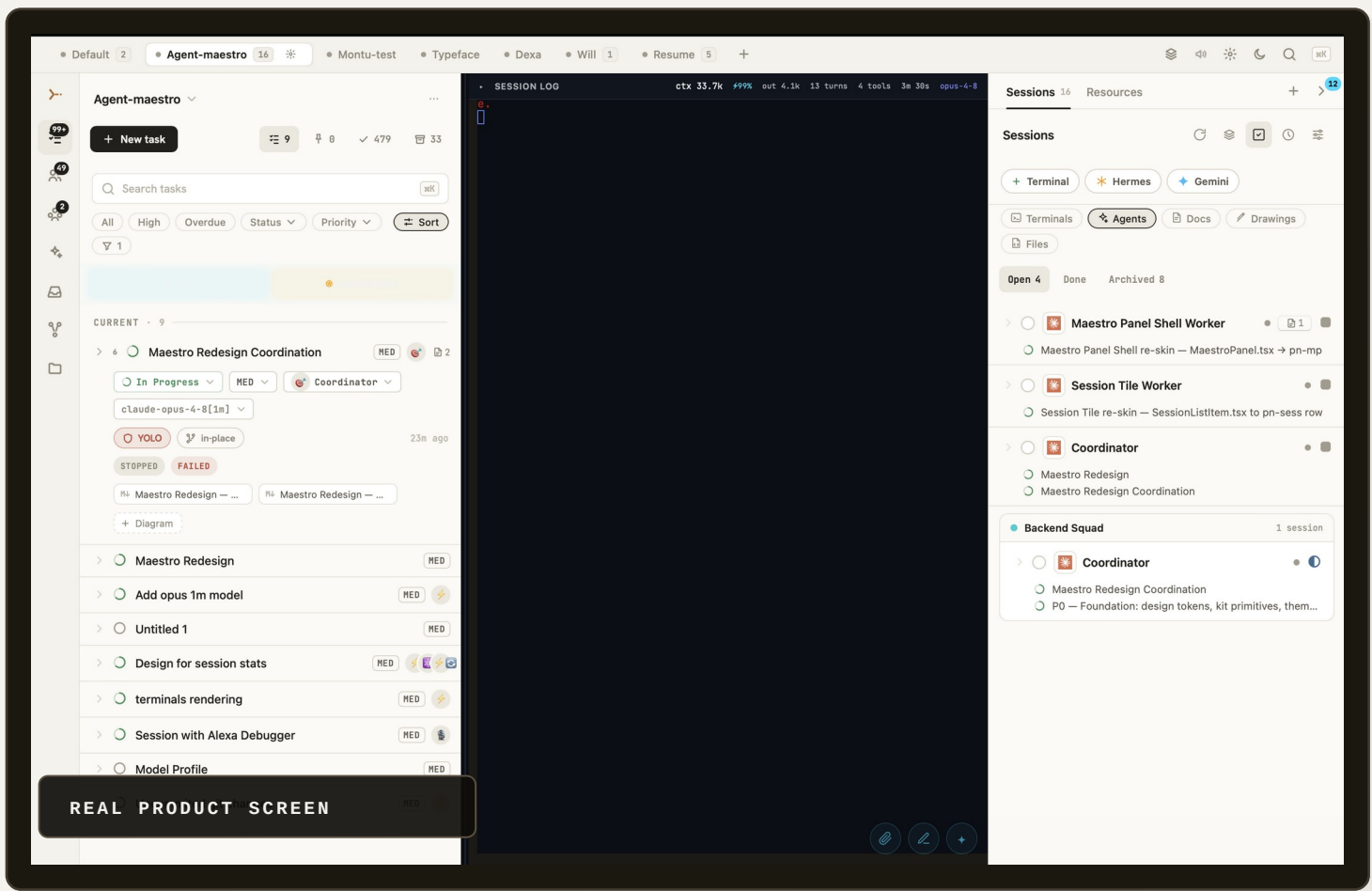
Execution and collaboration stay distinct—but exchange the same durable objects.



The durable asset is not the chat transcript. It is the reusable work object—and the proof attached to it.

The MVP already has a meaningful strategic shape.

These are not roadmap slogans; they are visible product primitives.



01 Ideas → tasks

Durable capture, hierarchy, references, acceptance.

02 One workspace

Multiple tools, projects, terminals, and sessions.

03 Team members

Reusable identity, runtime, permissions, memory, skills.

04 Deliberate orchestration

Worker, queue, tree, batch, DAG, and teams.

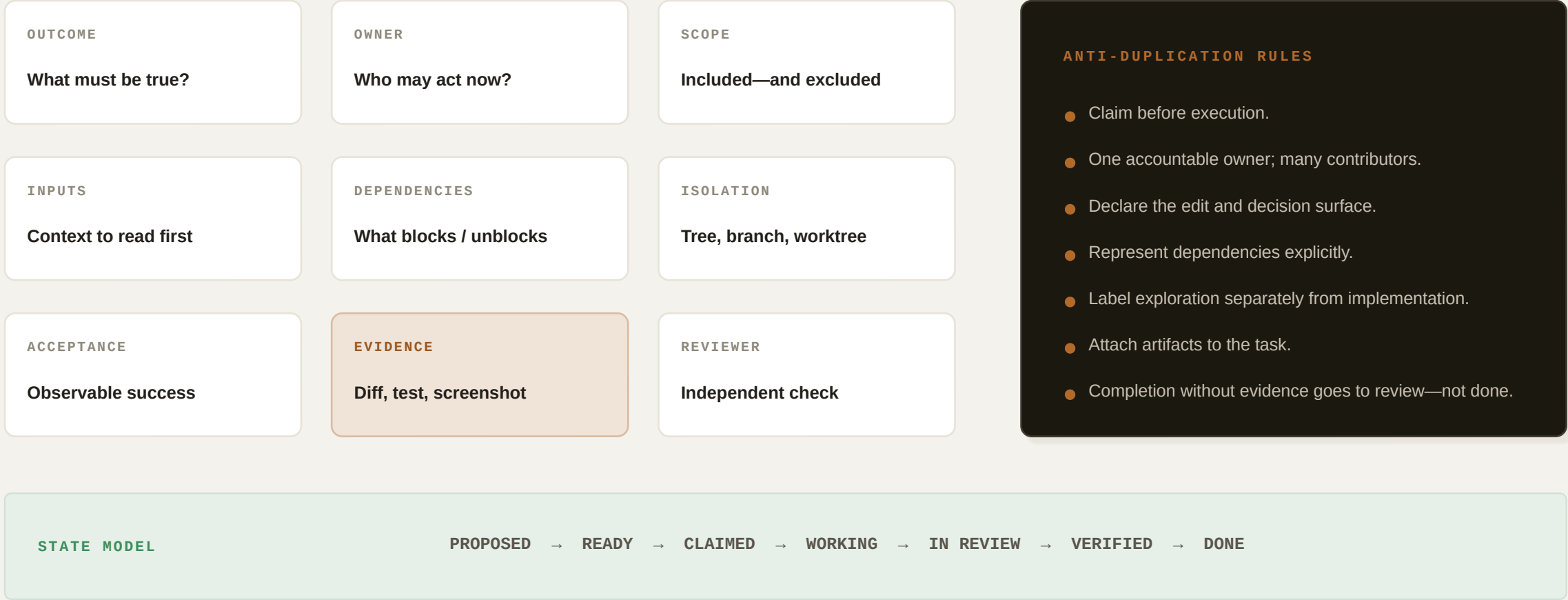
05 Collab + Git

Shared work objects around the code system of record.

BUILT NOW

A good task is a contract—not a prompt.

Make ownership, boundaries, dependencies, and proof visible before execution starts.



Assignment routes the work. Verification earns “done.”

Do not ask one agent to be planner, implementer, and judge by default.

ASSIGNMENT PROTOCOL

- 1

Classify the task
explore · design · build · debug · review
- 2

Match the specialist
identity · skills · permissions · memory
- 3

Choose the runtime
best-fit capability—not favorite vendor
- 4

Choose isolation
shared tree · worktree · sandbox · read-only
- 5

Set checkpoints
plan approval for risky or ambiguous work

VERIFICATION PROTOCOL

- | | |
|--|---|
| <div>READ</div> <div>Outcome + acceptance</div> | <div>INSPECT</div> <div>Artifact, diff, or design</div> |
| <div>RUN</div> <div>Named checks and tests</div> | <div>COMPARE</div> <div>References + contract</div> |
| <div>RECORD</div> <div>Findings as executable deltas</div> | <div>DECIDE</div> <div>Approve · return · split follow-up</div> |

DONE REQUIRES EVIDENCE. A completion message is a claim. The evidence bundle is the proof.

Onboard users to one closed loop—not every feature.

First-run promise: bring one real task; leave with one verified result and a reusable setup.

- 1 Connect a repo**
Detect working directory + Git remote.
- 2 Choose a starter lane**
Bug · feature · review · design exploration.
- 3 Define with a template**
Outcome · scope · acceptance · evidence.
- 4 Pick a specialist + runtime**
Default safely; explain the tradeoff.
- 5 Start in the right isolation**
Worktree for edits; read-only for review.
- 6 Verify and save the win**
Approve evidence; preserve memory or template.

GET WORK DONE ON THE MOVE



Team members are Skills++.

A skill is reusable instruction. A team member is a reusable operating identity.

REUSABLE ROLE

Frontend Lead

“Own interface quality, preserve the design system, and require visual evidence before completion.”

IDENTITY

How it thinks

RUNTIME

Agent + model

PERMISSIONS

What it may do

MEMORY

What survives

SKILLS

Playbooks loaded

POSITION

Where it sits

Members · 49

Agent-maestro > New team member

Name — e.g. Frontend Dev

AVATAR

ROLE ★

MODE

SCOPE

e.g. frontend specialist, tester

Worker

Orchestrator

Project

IDENTITY

Describe this team member's persona, expertise, and how they should approach tasks...

AGENT & MODEL

Claude

Codex

Gemini

Hermes

sonnet

Accept edits

Capabilities

Skills

Sound

Custom model

Skills

Memory

Hierarchy

Turn design feedback into an executable improvement loop.

Separate exploration, decision, implementation, review, and learning.



REVIEW AXES

- User outcome
- Design-system consistency
- Accessibility
- Visual evidence
- Information hierarchy
- All states + breakpoints
- Correctness + regressions

EVERY FINDING

- | | |
|--------------|---------------------------|
| WHAT | Observable issue |
| WHY | Violated need or contract |
| FIX | Smallest clear correction |
| PROOF | How review will verify |

Collaboration should share executable context—not just messages.

Repo-scoped spaces become the Agentic Product Management layer.

BUILT NOW

A credible foundation

- Auth + repo discovery
- Public / private spaces
- Channels + messaging
- Members roster
- Share tasks, personas, spells
- Files + docs
- Push / pull provenance
- Deployed security rules + tests

THE SHARED STREAM



Rhea

Design contract attached



Arun

Pulled task #142



Code Reviewer

3 findings · 1 blocker

@CodeReviewer review PR #42

NEXT

The killer flow

@agent mention



INVOKE



WORK



POST RESULT



CONTINUE

Humans and agents share the same work context, attribution, and history.

Native orchestration is improving. That validates the category.

Maestro cannot win on “they do not orchestrate.” It must win on neutrality, continuity, and workflow discipline.

PRODUCT TYPE	MULTI-AGENT	CROSS-PROVIDER	DURABLE WORK	VERIFICATION	COLLAB AROUND WORK
Provider-native	strong	one	provider	growing	provider
IDE-native agent	growing	limited	editor	diff-led	secondary
Generic PM / chat	none	agnostic	human	manual	strong
MAESTRO	integrates	YES	SHARED GRAPH	EVIDENCE GATE	EXECUTABLE
STRATEGIC RULE	Integrate provider-native strengths. Own the shared plan, ownership, evidence, and cross-tool memory.				

Sources: Anthropic Claude Code agent teams / parallel agents · OpenAI Codex app · Nous Hermes Agent docs (accessed July 2026)

The moat is accumulated workflow memory—not another adapter.

Every completed loop should make the next loop easier, safer, and more reusable.



Adapters widen the network. The reusable organization inside the network creates switching value.

Community before cloud. Outcomes before monetization.

The immediate goal is 100–1,000 retained builders who repeatedly complete real work.

100 → 1,000

retained power users

The goal is not reach. It is repeated, verified work—and visible proof that the workflow improves.

NO CLOUD RUSH

X / TWITTER

Founder build logs, 60-second workflow clips, honest failure reports.

WEEKLY MAESTRO RUN

One objective, task tree, agent mix, evidence, outcome.

WORKFLOW LIBRARY

Templates, team members, review rubrics, skills, spells.

OFFICE HOURS

Onboard live. Watch where users fail. Fix recurring blockers.

PUBLIC FEEDBACK LOOP

Show which user feedback changed the product and why.

LOCAL / FREE PATH

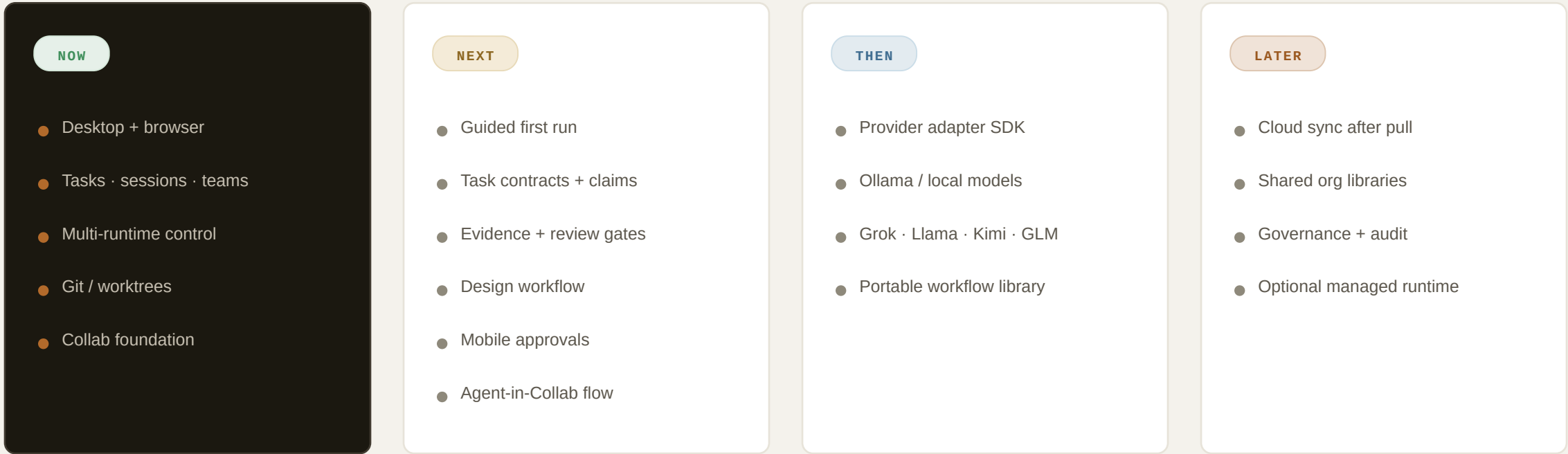
Ollama and curated models for the broader creator audience—later.

ACQUISITION HOOK

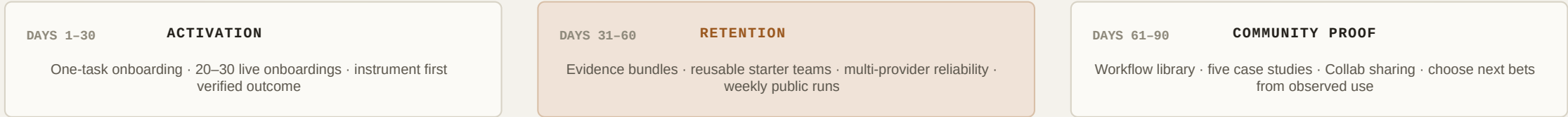
“If your AI team lives across six terminals and your memory, you have an operating-system problem.”

Improve the loop, widen the ecosystem, then earn the cloud.

Roadmap order follows user pull and retention—not feature count.



NINETY-DAY OPERATING PLAN



Measure verified progress. Design against the obvious risks.

Avoid vanity metrics that reward motion without outcomes.

NORTH STAR

VERIFIED
WORK LOOPS

per weekly active user

- Time to first verified outcome
- Week 1 / week 4 retained builders
- % tasks with owner + acceptance + evidence
- Templates / team members reused
- Users running more than one provider
- Outcome stories with before / after proof

RISKS → RESPONSES

Frontier bundling	→	Own cross-provider workflow; integrate native capabilities.
Complexity	→	Teach one closed loop; reveal power progressively.
Low-quality agent output	→	Make evidence, review roles, and checkpoints first-class.
Vague “everything” story	→	Keep the wedge: multi-agent software delivery for power users.
Integration churn	→	Explicit adapters, capability detection, graceful fallback.
Slack-clone drift	→	Tie every conversation to executable work objects.

Do not optimize for: total sessions · total tasks created · raw signups · token volume without outcomes

Lead with control. Prove it with the workflow.

The message should sound like a builder sharing a solved problem—not a category essay.

PRIMARY HEADLINE

**Orchestrate any agent.
Keep control of the work.**

Maestro turns ideas into owned, observable, verified work across Claude, Codex, Hermes—and whatever you use next.

FREEDOM OF TOOLS. DISCIPLINE OF PROCESS.

FOUNDER STORY

“I did not build Maestro because agents could not code. I built it because my process stopped scaling when I used more of them.”

COMMUNITY CTA

**Bring one painful workflow.
Run it through Maestro.
Show us where the system breaks.**

One wedge. One loop. One open control plane.

The decision summary.

WHO FIRST Power users of Claude, Codex, and Hermes already coordinating multiple sessions.	SURFACE PAIN Too many agents, terminals, projects, and partial contexts to track.	ROOT PROBLEM No shared system for owned, verifiable, reusable agentic work.
CATEGORY Agentic Product Management.	PRODUCT Tasks + agents + tools + Git + artifacts + review + memory + Collab.	UNIQUE Cross-provider freedom with disciplined process; team members as Skills++.
MOAT Accumulated workflow memory, reusable organization, and community artifacts.	GOAL NOW 100–1,000 retained builders. Improve hard. Publish proof.	SEQUENCE Perfect the loop → expand adapters/local models → release cloud after pull.

MAESTRO = THE OPEN CONTROL PLANE FOR AGENTIC PRODUCT WORK

Two operating templates to ship with the product.

The strategy becomes valuable when it changes daily behavior.

TEMPLATE 01

Executable task

OUTCOME	What will be true?
OWNER	One accountable actor
SCOPE	In / out
INPUTS	Read first
DEPENDENCIES	Blocks / unblocks
ISOLATION	Worktree / read-only
ACCEPTANCE	Observable checks
EVIDENCE	Artifact / diff / test
REVIEWER	Independent judge

TEMPLATE 02

Review finding

WHAT

State the observable issue.

WHY

Name the violated user need, contract, or system rule.

FIX

Describe the smallest clear correction.

PROOF

Define how the reviewer will verify it.

Review feedback becomes an owned improvement queue—not another conversation to remember.

Evidence base and claim discipline.

Built, next, and later are intentionally separated throughout the deck.

INTERNAL PRODUCT EVIDENCE

- README.md — desktop, CLI, browser, tasks, sessions, local-first architecture
- CLAUDE.md — domain entities, runtimes, team members, skills, Git
- docs/workflow-strategies.md — queue, tree, batch, DAG coordination
- docs/TEAM_MEMBERS_UNDERSTANDING.md — identities, memory, permissions, hierarchy
- docs/collab-design/design-spec/00-OVERVIEW.md — Collab purpose and current status
- docs/COLLAB_SPACE_VERIFICATION_LOG.md — deployed rules and test evidence
- docs/collab-design/design-spec/08-FULL-VISION-ROADMAP.md — future Collab features
- maestro-mobile/ + Maestro Design System - mobile/ — mobile foundation

CURRENT CATEGORY EVIDENCE

Anthropic · Claude Code parallel agents and agent teams

code.claude.com/docs/en/agents · [/agent-teams](#)

OpenAI · Introducing the Codex app

openai.com/index/introducing-the-codex-app/

Nous Research · Hermes Agent documentation

STATUS LEGEND

BUILT NOW

Implemented product foundation

NEXT

Near-term strategic priority

LATER

Expansion after demonstrated pull